30-Customer Assortment Game 2

IEOR 145/245 · GROUP SIMULATION EXERCISE · SECOND GAME (SIMULATOR RECORD DOWNLOADED OCTOBER 25, 2026)

We played the simulation a second time with the same model and policy as in the first game, and again sold all 18 units for the maximum possible revenue of **$270**, this time selling the last unit to customer 25 of 30. Every one of the 25 assortments we showed was exactly what `code/play.py` recommended. Unlike the first game, product 3 ($20) sold out first, after only 14 customers. This happened because type A customers arrived early and in large numbers, not because of any change in the policy.

REVENUE	ALL UNITS SOLD AFTER	FORECAST BEFORE THE GAME	CHANCE OF $270 WITH THESE CUSTOMERS
$270 the maximum	**customer 25 of 30** 5 to spare	**$268.7** expected under our policy	**93.3%** vs. 57.4% for the greedy rule from class

The game

- Three products: product 1 at $10, product 2 at $15, product 3 at $20, with 6 units of each and no restocking.
- 30 customers arrive one at a time. Each is type A, B or C, and we see the type when the customer arrives.
- For each customer we choose which in-stock products to show (the *assortment*). The customer buys at most one of them or walks away.
- Goal: maximize total revenue. With 18 units for 30 customers, the $270 maximum means selling every unit.

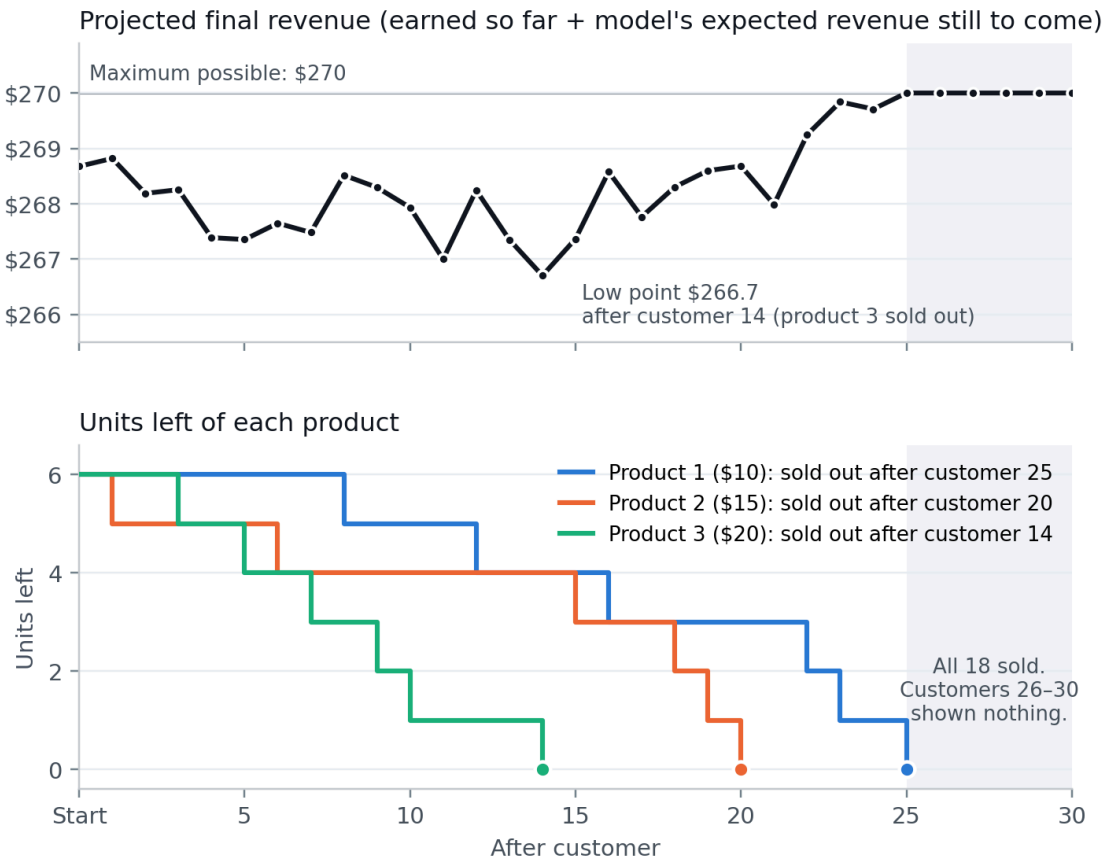
The model and policy (unchanged from game 1)

We used the same model, code and settings as in the first game, with no re-tuning in between. For each customer type we fitted a multinomial logit (MNL) choice model to the 63,000 past customers in the training data (Lecture 14A).

TYPE	BUYS PRODUCT 1 WHEN SHOWN ALONE	BUYS PRODUCT 2 WHEN SHOWN ALONE	BUYS PRODUCT 3 WHEN SHOWN ALONE	MNL WEIGHTS v_1, v_2, v_3	IN SHORT
A	18%	16%	**62%**	0.20, 0.19, 1.59	Buys product 3 or nothing
B	49%	**87%**	51%	1.01, 6.95, 1.01	Strongly prefers product 2
C	66%	67%	68%	2.01, 2.07, 2.04	Buys whatever is shown

A dynamic program then chooses, for every combination of customer number, remaining stock and arriving type, the assortment that maximizes expected total revenue. Each unit is valued at its *opportunity cost*: what it is expected to earn if kept for a later customer. The policy plans for types arriving at random in the training data's mix (11 A : 7 B : 12 C out of 30). In practice it holds product 3 back for type A, keeps product 2 for type B, and uses type C to sell whatever is piling up, especially product 1. The method, model checks and robustness tests are in the first game's report (`report/Assortment_Simulation_Report.pdf`) and in `notebooks/RM_assortment_model.ipynb`.

How the game went



Top: revenue earned so far plus the model's expected revenue from the remaining customers, after each customer. Bottom: units left of each product. Customers 26–30 (shaded) arrived after everything was sold.

The projection stayed between $266.7 and $269.8 the whole game, never more than $3.3 below the maximum. The game had three phases.

Customers 1–14: product 3 goes to type A. Eight of the first 14 customers were type A and none were type B. Each of these type A customers was shown `{1,2,3}`. They bought product 3 six times (customers 3, 5, 7, 9, 10 and 14) and walked away twice. Type C customers were never shown product 3: they saw `{1,2}` or `{1}`, which kept product 3 for type A, and bought two units of product 2 and two of product 1. The last unit of product 3 sold to customer 14, leaving stock of (4, 4, 0) and $170 of revenue.

Customers 15–20: products 1 and 2 left. With 8 units left for 16 customers, type C customers 15 and 16 were shown `{1,2}` and bought one of each. Customer 17 was type A, and with product 3 gone there was nothing they were likely to buy. They were shown `{1,2}`, which they walk away from 72% of the time, and they did. Then three type B customers arrived in a row (18–20) and bought the last three units of product 2.

Customers 21–25: clear product 1. Three units of product 1 were left for 10 customers, so everyone was shown `{1}`. Type B customers 22 and 23 bought it (B buys product 1 about half the time when it is the only product shown), and type C customer 25 bought the last unit. Customers 26–30 arrived after everything was sold.

Game log

CUSTOMER	TYPE	STOCK ON ARRIVAL	SHOWN	P(NO BUY)	OUTCOME	REVENUE AFTER	PROJECTED ON ARRIVAL
1	C	(6, 6, 6)	{1,2}	0.20	product 2 ($15)	$15	$268.7
2	C	(6, 5, 6)	{1}	0.33	no purchase	$15	$268.8
3	A	(6, 5, 6)	{1,2,3}	0.34	product 3 ($20)	$35	$268.2
4	A	(6, 5, 5)	{1,2,3}	0.34	no purchase	$35	$268.3
5	A	(6, 5, 5)	{1,2,3}	0.34	product 3 ($20)	$55	$267.4
6	C	(6, 5, 4)	{1,2}	0.20	product 2 ($15)	$70	$267.4
7	A	(6, 4, 4)	{1,2,3}	0.34	product 3 ($20)	$90	$267.6
8	C	(6, 4, 3)	{1}	0.33	product 1 ($10)	$100	$267.5
9	A	(5, 4, 3)	{1,2,3}	0.34	product 3 ($20)	$120	$268.5
10	A	(5, 4, 2)	{1,2,3}	0.34	product 3 ($20)	$140	$268.3
11	A	(5, 4, 1)	{1,2,3}	0.34	no purchase	$140	$267.9
12	C	(5, 4, 1)	{1,2}	0.20	product 1 ($10)	$150	$267.0
13	C	(4, 4, 1)	{1,2}	0.20	no purchase	$150	$268.2
14	A	(4, 4, 1)	{1,2,3}	0.34	product 3 ($20)	$170	$267.4
15	C	(4, 4, 0)	{1,2}	0.20	product 2 ($15)	$185	$266.7
16	C	(4, 3, 0)	{1,2}	0.20	product 1 ($10)	$195	$267.4
17	A	(3, 3, 0)	{1,2}	0.72	no purchase	$195	$268.6
18	B	(3, 3, 0)	{1,2}	0.11	product 2 ($15)	$210	$267.8
19	B	(3, 2, 0)	{1,2}	0.11	product 2 ($15)	$225	$268.3
20	B	(3, 1, 0)	{1,2}	0.11	product 2 ($15)	$240	$268.6
21	C	(3, 0, 0)	{1}	0.33	no purchase	$240	$268.7
22	B	(3, 0, 0)	{1}	0.50	product 1 ($10)	$250	$268.0
23	B	(2, 0, 0)	{1}	0.50	product 1 ($10)	$260	$269.2
24	C	(1, 0, 0)	{1}	0.33	no purchase	$260	$269.8
25	C	(1, 0, 0)	{1}	0.33	product 1 ($10)	$270	$269.7
26–30	A, B, B, C, A	(0, 0, 0)	nothing in stock	–	–	$270	$270.0

Stock is units of products 1, 2, 3 when the customer arrived. "Shown" is the assortment in the simulator's record; it matches `play.py`'s recommendation for every customer. P(no buy) is the fitted model's chance that the customer walks away from the assortment shown. "Projected on arrival" is revenue so far plus the model's expected revenue from this customer onward, the same number `play.py` prints. Source: `simulation_results/LIVE-RHR5BR_game2_assortment_simulation_history.csv`, replayed through the policy; the replayed stock and revenue match the record on every row.

Why product 3 sold out first

In the first game product 3 was the last product to sell out (after customer 27). This time it was the first, after only customer 14. The policy did not push product 3 harder. The difference was who arrived:

- **More type A customers early than usual.** Type A buys product 3 or nothing. 8 of the first 14 customers were type A, against 5.1 expected; at the data's mix, 8 or more type A customers in the first 14 has a 9.7% chance.
- **No type B customers early.** None of the first 14 customers were type B (3.3 expected; 2.4% chance). Type B is the main buyer of product 2, so only 4 of the 12 units of products 1 and 2 sold in the first 14 customers.
- **Type A customers bought more often than predicted.** They bought product 3 on 6 of 8 offers (75%), against the model's 53% for `{1,2,3}`. Six or more out of eight has a 19.4% chance.

- **All 6 units went to type A, the intended buyers.** No type B or C customer was ever shown product 3. Product 3 sold at full price to exactly the customers the policy reserves it for, so selling out early cost nothing.

Before the game, product 3 sold out within the first 14 customers in only 4.3% of 50,000 simulated games. Given this game's actual order of customer types, the chance rises to 20.0%. Product 3 selling out *first* is less unusual. In simulated games that sell every unit, product 3 is the first to sell out 24.5% of the time; product 2 is first in 50.0%. The order in which products run out does not affect revenue. What matters is whether every unit sells. When a game ends with stock left over, it is usually product 1 (86% of such games), the product type A rarely buys.

Policy or luck?

Customer behavior matched the model. 7 of the 25 customers who were offered something walked away, against 7.6 expected. **The mix of types was exactly as planned.** Over all 30 customers, 11 were type A, 7 type B and 12 type C, the same as the training data's 11 : 7 : 12. Only the order was unusual.

To separate the policy from luck, we replayed this game's 30 customer types, in order, through each policy and computed the exact expected revenue and the exact chance of reaching $270.

POLICY, SAME 30 CUSTOMER TYPES	EXPECTED REVENUE	CHANCE OF $270
Dynamic program (ours, as played)	**$268.89**	**93.3%**
Show all in stock	$268.47	89.9%
Inventory balancing (Golrezaei et al. 2014)	$266.71	80.0%
Myopic greedy (Lecture 14B)	$261.77	57.4%
Clairvoyant planner (knows all 30 types in advance)	*$269.09*	*94.2%*

Exact values by policy evaluation. Customer behavior uses the training data's observed choice frequencies; using the fitted MNL instead changes each value by at most $0.65 and 2.7 percentage points. Before the game (types unknown), the same policy had an expected revenue of $268.57 and a 91.4% chance of $270.

With these customers, our policy had a 93.3% chance of reaching $270, within one point of a planner who knew every customer's type in advance (94.2%). The greedy rule from class would have reached $270 only 57.4% of the time. The early run of type A customers did not hurt us: our chance of $270 with this order of customers was higher than before the game (93.3% vs. 91.4%). As in game 1, the $270 was the likely result of the policy, not luck.

Game 1 and game 2 side by side

	GAME 1	GAME 2
Revenue	$270	$270
All units sold after customer	27	25
Followed `play.py`	Customers 3–27 (1–2 were shown `{2,3}`)	All 25 customers
Customers who walked away (expected)	9 (7.4)	7 (7.6)
Order products sold out	product 2 (after 20), product 1 (22), product 3 (27)	product 3 (after 14), product 2 (20), product 1 (25)
Chance of $270 with that game's customers: ours / clairvoyant	94.0% / 95.1%	93.3% / 94.2%
Chance of $270 with that game's customers: greedy	50.7%	57.4%

Game 1 values are from the first report. The simulator stopped recording game 1 after customer 27, so its replay drew customers 28–30 at random from the data's mix. Game 2's record includes all 30 customers.

The same policy produced very different games. In game 1 type A customers came late, so product 3 sold last; in game 2 they came early, so it sold first. In both games the policy gave each product to the type that buys it at full price, and both reached $270 with over 90% probability given the customers who arrived.

Limitations

- Two games are two random draws. Over a full 30-customer game, revenue under the best policy still has a standard deviation of about $5.50. Before the game, there was a 56.5% chance of selling out by customer 25 or earlier.
- The policy assumes customers behave like the training data. Both games are consistent with this (16 walk-aways against 15.0 expected across the two games), but two games are too few to test it.
- The policy plans for types arriving at random in the data's mix. Game 2's early run of type A customers is the kind of order it is not tuned for, and it still came within one point of the clairvoyant planner.

Reproduce: `code/rm_model.py` (model, dynamic program, baselines and exact evaluation), `code/play.py` (the live helper used in this game), `notebooks/RM_assortment_model.ipynb` (full analysis) and `simulation_results/LIVE-RHR5BR_game2_assortment_simulation_history.csv` (this game's simulator record). Simulated percentages use 50,000 games with the policy above, customer behavior from the training data's observed frequencies, and types drawn at random in the data's mix.